SIMATS ENGINEERING

CO2_AT3 Error Analysis Task

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 20%

Sl. No	Questions	Bloom's Level
1	A tester designs test cases for an online shopping system using equivalence partitioning but incorrectly groups input ranges (e.g., valid and invalid inputs mixed in one class). Additionally, boundary values are not considered. Analyze the given test design and identify all errors in equivalence class partitioning and boundary value analysis. Examine the root causes of these mistakes with proper reasoning. Propose corrected test cases with proper classification and boundary coverage. Justify how your corrections improve defect detection effectiveness.	3
2	A QA engineer performs random testing on a banking application and claims high coverage without using any structured techniques. Critical defects in transaction limits and boundary conditions are missed. Evaluate the testing approach and identify errors in strategy and coverage assumptions. Analyze the root causes behind missing critical defects. Suggest appropriate black box testing techniques to correct the approach. Justify how these techniques ensure better test adequacy and reliability.	3
3	A developer performs white box testing but creates an incomplete control flow graph, missing decision nodes and loops, resulting in poor path coverage. Analyze the errors in the control flow graph and path testing approach. Identify the root causes for inadequate test coverage. Propose corrections by constructing a proper control flow representation and identifying all independent paths. Justify how improved path testing ensures better coverage and defect detection.	5

4	A testing team reports 100% statement coverage for a module but fails to detect logical errors in conditional statements. Evaluate the adequacy of the testing performed and identify gaps in coverage criteria. Analyze why statement coverage alone was insufficient. Recommend improved white box testing strategies such as branch or path coverage. Justify how the revised approach enhances overall test effectiveness and error detection.	5
5	A tester uses Equivalence Class Partitioning for an age-validation field (0-100) but defines only two classes: Negative numbers and Positive numbers. Analyze the errors in this partitioning strategy. Identify why this fails to account for valid vs. invalid age limits and the absence of Boundary Value Analysis. Propose a corrected set of partitions and justify how they ensure more robust input validation.	3
6	A developer claims full Condition Coverage for a complex `if(A && B	
7	A QA team applies Decision Table Testing for a discount system but misses several combinations of "Premium Member" and "Coupon Applied," leading to price calculation errors. Analyze the errors in the construction of the decision table. Identify why the current approach is incomplete. Propose a revised, exhaustive decision table and justify its effectiveness in preventing production defects.	3
8	During unit testing of a looping structure, a tester checks only the first and last iterations (n=1 and n=max). Defects occurring at n=0 and n=max+1 are missed. Evaluate the errors in this Loop Testing strategy. Analyze the root causes behind these omissions. Recommend a corrected approach for testing simple and nested loops, and justify how it ensures path integrity.	5
9	An engineer utilizes Error Guessing as the primary testing technique for a safety-critical braking system, ignoring structured black box methods. Identify the errors in relying solely on ad-hoc techniques for high-risk modules. Analyze the impact on test adequacy. Suggest a combination of BVA and State Transition Testing to fix the strategy, justifying the increased safety assurance.	5

10	A tester designs a Control Flow Graph (CFG) for a module but fails to represent the "else" branches of multiple nested if statements, leading to 100% statement coverage but 0% branch coverage on error paths. Analyze the errors in graph construction. Identify the root causes for the missing nodes. Propose a corrected CFG and independent paths to ensure branch adequacy.	5

Code of Conduct:

I __K.Mathivadhani(192311424)__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:Mathivadhani.K

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

1. Errors in Equivalence Class Partitioning and Boundary Value Analysis

A. Introduction

Equivalence Class Partitioning (ECP) and Boundary Value Analysis (BVA) are black box testing techniques used to reduce the number of test cases while maintaining effective coverage. ECP divides inputs into groups expected to behave similarly, whereas BVA tests values near the limits of those groups.

B. Errors in the Existing Test Design

- Valid and invalid inputs are incorrectly grouped into the same equivalence class.
- Minimum and maximum input limits are not identified.
- Values immediately below and above the limits are ignored.
- Test cases are selected without considering the expected output for each class.
- Random selection of values may leave important defects undetected.

C. Root Causes

These mistakes occur because the tester has not properly analyzed the system requirements and input constraints. Incorrect partitioning assumes that all values behave similarly, even when valid and invalid inputs require different processing. Ignoring boundary values also overlooks common errors such as incorrect comparison operators and off-by-one mistakes.

D. Corrected Test Cases

Consider an online shopping quantity field that accepts values from 1 to 100.

Test case	Input	Classification	Expected result
TC1	-1	Invalid class	Reject input
TC2	0	Invalid boundary	Reject input
TC3	1	Minimum valid boundary	Accept input
TC4	50	Valid class	Accept input
TC5	100	Maximum valid boundary	Accept input
TC6	101	Invalid boundary	Reject input

The equivalence classes are negative quantities, zero, valid quantities from 1 to 100, and quantities greater than 100.

E. Conclusion

Correct ECP and BVA improve test effectiveness by ensuring that every meaningful input class and critical boundary is tested. This helps detect validation errors, incorrect limit checks, and unexpected system behavior.

2. Evaluation of Random Testing in a Banking Application

A. Introduction

Random testing selects inputs without following a predefined systematic strategy. Although it may discover unexpected defects, relying only on random testing is insufficient for a banking application because transaction limits, balance validation, and financial calculations require predictable and measurable coverage.

B. Errors in the Testing Approach

- Test inputs are selected without systematic classification.
- Important transaction boundaries may never be tested.
- High test-case quantity is incorrectly treated as proof of high coverage.
- Expected results and requirements may not be defined for every test.
- Critical combinations, such as insufficient balance and maximum withdrawal limits, may be overlooked.

C. Root Causes

The primary cause is the absence of requirement-based test design. Random testing does not guarantee that specific boundaries, decision outcomes, or transaction states will be exercised. Without coverage metrics and a traceability matrix, the tester cannot demonstrate that all important requirements have been verified.

D. Recommended Black Box Techniques

- Equivalence Class Partitioning: Divide transaction amounts into valid amounts, zero or negative amounts, and amounts exceeding the permitted limit.
- Boundary Value Analysis: Test amounts at the minimum and maximum limits and immediately below and above them.
- Decision Table Testing: Test combinations of sufficient balance, transaction limit, account status, and authentication status.
- State Transition Testing: Verify transitions such as initiated, pending, successful, failed, and reversed transactions.
- Error Guessing: Supplement structured techniques with likely failure scenarios such as duplicate transactions.

For example, if the withdrawal limit is ₹20,000, test ₹19,999, ₹20,000, and ₹20,001, subject to the system's stated rules.

E. Conclusion

Random testing should be used as a supplementary technique rather than the primary strategy. Combining ECP, BVA, decision tables, and state transition testing provides measurable coverage and improves banking transaction reliability.

3. Errors in Control Flow Graph and Path Testing

A. Introduction

White box testing examines the internal structure of a program. A Control Flow Graph (CFG) represents statements, decisions, loops, and possible execution paths. An incomplete CFG can cause important branches to remain untested, reducing path coverage and defect detection.

B. Errors in the Existing CFG

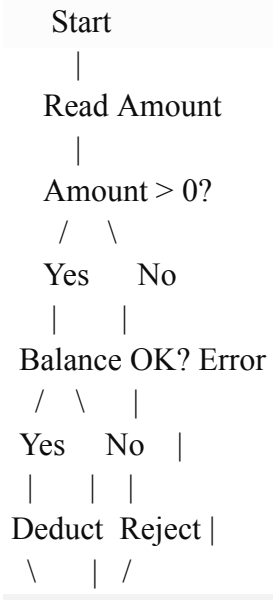
- Decision nodes for if, else if, and other conditions are missing.
- False branches are omitted.
- Loop entry and exit conditions are not represented.
- Some control flow edges are incorrectly connected.
- Independent execution paths are not identified completely.

C. Root Causes

The main causes are incomplete code analysis, misunderstanding of branching logic, and failure to trace loop conditions. If a CFG contains only the successful execution route, errors in alternative paths cannot be identified through path-based testing.

D. Corrected CFG and Independent Paths

Consider this example:



The independent paths are:

- Path 1: Start → Read amount → Positive amount → Balance sufficient → Deduct amount → End.
- Path 2: Start → Read amount → Positive amount → Balance insufficient → Reject transaction → End.
- Path 3: Start → Read amount → Non-positive amount → Error → End.

These paths cover the true and false outcomes of both decisions.

E. Conclusion

A complete CFG and independent path analysis improve branch and path coverage. They help detect errors in validation, transaction processing, and alternative execution routes that may remain hidden during ordinary testing.

4. Limitations of 100% Statement Coverage

A. Introduction

Statement coverage measures the percentage of executable statements executed during testing. Although 100% statement coverage indicates that every statement has run, it does not guarantee that every decision outcome or logical condition has been tested.

B. Gaps in the Existing Coverage

- False branches may remain untested.
- Compound conditions may contain logical defects.
- Incorrect Boolean operators may go unnoticed.
- Alternative execution paths may not be exercised.
- Exceptional and error-handling paths may be omitted.

C. Why Statement Coverage Is Insufficient

Consider the following code:

```
if (age >= 18 && hasID)
   AllowEntry();
else
   DenyEntry();
```

If the tester uses only an input where age >= 18 is true and hasID is true, the AllowEntry() statement executes. However, the false branch has not been tested. Additional inputs are needed to exercise both outcomes and the individual conditions.

D. Improved White Box Testing Strategies

- Branch coverage: Execute both true and false outcomes of each decision.
- Condition coverage: Make every individual Boolean condition true and false.

- Path coverage: Test the feasible execution paths through the program.
- Modified Condition/Decision Coverage (MC/DC): Demonstrate that each condition can independently affect the final decision.
- Loop testing: Test zero, one, and multiple iterations where applicable.

For the example, use an adult with ID, an adult without ID, and a minor with ID. These cases exercise the main decision outcomes and individual conditions.

E. Conclusion

Statement coverage alone is not sufficient for reliable testing. Adding branch, condition, and path coverage provides stronger evidence of correctness and improves the detection of logical errors.

5. Correct Equivalence Classes for Age Validation

A. Introduction

Equivalence Class Partitioning divides input values into groups that are expected to produce similar results. For an age-validation field accepting ages from 0 to 100, dividing inputs only into negative and positive numbers is incorrect because it combines valid and invalid values into the same class.

B. Errors in the Existing Partitioning

- All positive values are grouped together, including valid ages and ages greater than 100.
- The valid value zero is not properly distinguished from negative values.
- The maximum permitted age is not identified.
- Boundary values such as 0, 1, 99, 100, and 101 are not considered.
- The expected behavior for invalid inputs is not clearly defined.

C. Corrected Equivalence Classes

Assuming the system accepts integer ages from 0 through 100:

- Class 1: Ages below 0 — invalid.
- Class 2: Ages from 0 to 100 — valid.
- Class 3: Ages above 100 — invalid.
- Class 4: Non-numeric inputs, such as letters — invalid.
- Class 5: Empty or missing input — invalid, unless the requirements specify otherwise.

D. Boundary Value Test Cases

Test case	Age input	Expected result
TC1	-1	Reject
TC2	0	Accept

TC3	1	Accept
TC4	50	Accept
TC5	99	Accept
TC6	100	Accept
TC7	101	Reject
TC8	"abc"	Reject
TC9	Empty input	Reject

These tests cover valid and invalid equivalence classes as well as the boundaries of the permitted range.

E. Conclusion

Correct ECP and BVA improve age validation by separating valid and invalid values and testing the limits explicitly. This reduces errors caused by incorrect comparisons and ensures that the system follows the stated requirements.

6. Condition Coverage for a Complex If Statement

A. Introduction

Condition coverage is a white box testing technique that ensures each individual Boolean condition in a decision evaluates to both true and false at least once. It is especially useful for identifying errors in compound decisions involving logical operators such as AND (&&) and OR (||).

B. Example Program

```
if (A && B)
   ExecuteAction();
else
   DisplayError();
```

Here, the overall decision is true only when both A and B are true.

C. Testing Requirements

To achieve condition coverage:

- Condition A must evaluate to true at least once.

- Condition A must evaluate to false at least once.
- Condition B must evaluate to true at least once.
- Condition B must evaluate to false at least once.

D. Test Cases

Test case	A	B	Overall decision
TC1	True	True	True
TC2	False	True	False
TC3	True	False	False

These three tests make both conditions true and false. They also exercise both outcomes of the overall decision.

E. Conclusion

Condition coverage helps identify defects in Boolean expressions that statement coverage may miss. For high-risk decisions, Modified Condition/Decision Coverage (MC/DC) can provide stronger assurance by showing that each condition independently affects the overall result.

7. Decision Table Testing for a Discount System

A. Introduction

Decision Table Testing is a black box testing technique used to verify combinations of conditions and their corresponding actions. In a discount system, membership status and coupon usage may influence the final price. Missing combinations can cause incorrect discounts or billing errors.

B. Errors in the Existing Decision Table

- Some combinations of premium membership and coupon usage are missing.
- Discount rules may overlap or conflict.
- The order of applying discounts is not specified.
- Invalid or expired coupons may not be considered.
- Expected prices are not calculated and verified for every rule.

C. Root Causes

These errors occur when the tester does not enumerate all combinations of relevant conditions. With two Boolean conditions—premium membership and coupon application—there are four basic combinations. Omitting any combination leaves part of the decision logic untested.

D. Corrected Decision Table

Assume premium members receive a 10% discount and a valid coupon provides an additional 5% discount. For this example, the discounts are additive and calculated from the original price.

Rule	Premium member?	Valid coupon?	Expected discount
R1	Yes	Yes	15%
R2	Yes	No	10%
R3	No	Yes	5%
R4	No	No	0%

Additional tests should cover expired coupons, invalid coupon codes, zero-priced items, and the minimum purchase amount if the system has one.

E. Conclusion

A complete decision table ensures that every combination of membership and coupon conditions is tested. Verifying the expected final price for each rule helps prevent incorrect discount calculations and production billing defects.

8. Errors in Loop Testing Strategy

A. Introduction

Loop testing is a white box technique used to verify the correctness of loops and their iteration boundaries. Testing only the first and maximum iterations is insufficient because defects may occur when a loop executes zero times or one more time than the permitted maximum.

B. Errors in the Existing Strategy

- Zero iterations ($n = 0$) are not tested.
- The maximum plus one iteration ($n = max + 1$) is not tested.
- Loop termination conditions may contain off-by-one errors.
- Nested loops may not be tested independently.
- Infinite loops and incorrect counter updates may remain undetected.

C. Root Causes

The tester assumes that testing the first and last permitted iterations is sufficient. However, incorrect loop conditions such as < instead of <= may cause failures at boundaries. Missing zero-iteration testing can also hide errors when the loop should be skipped entirely.

D. Corrected Loop Testing Strategy

For a loop with a permitted maximum of max iterations, test:

- Zero iterations: n = 0, to verify that the loop is skipped correctly.
- One iteration: n = 1, to check basic execution.
- Typical iterations: A normal intermediate value.
- Maximum iterations: n = max, to verify the upper permitted limit.
- Maximum plus one: n = max + 1, to verify rejection or safe handling when the input exceeds the limit.

For nested loops, test the inner loop at zero, one, and multiple iterations while varying the outer loop. Then test combinations of both loops and check termination behavior.

E. Conclusion

A systematic loop-testing strategy detects boundary errors, incorrect termination conditions, and repeated-execution defects. Testing nested loops at multiple iteration counts improves path integrity and reduces the risk of infinite loops and incorrect results.

9. Error Guessing in a Safety-Critical Braking System

A. Introduction

Error Guessing relies on a tester's experience to anticipate likely defects. Although it can identify unusual failures, using it as the only testing technique for a safety-critical braking system is inadequate because braking behavior must be verified systematically under normal, boundary, and abnormal conditions.

B. Errors in the Existing Approach

- Test cases depend mainly on individual experience.
- Brake-response boundaries may be missed.
- Different operating states may not be covered.
- Critical combinations of speed, braking force, and sensor conditions may remain untested.
- There may be no measurable coverage criteria or traceability to safety requirements.

C. Risks and Root Causes

The main risk is incomplete testing of safety-critical behavior. Unexpected combinations of inputs can produce dangerous results even if common scenarios work correctly. Ad-hoc testing also makes it difficult to demonstrate that important safety requirements have been systematically verified.

D. Recommended Combined Testing Strategy

Boundary Value Analysis (BVA):

- Test minimum and maximum supported speed values.
- Test brake-force sensor readings at the lower and upper permitted limits.
- Test values immediately below, at, and above specified thresholds.
- Verify behavior for invalid or out-of-range sensor readings.

State Transition Testing:

- Normal driving → Braking requested.
- Braking requested → Brakes engaged.
- Brakes engaged → Vehicle stopped.
- Sensor failure → Safe fault-handling state.
- Braking system fault → Warning and defined fail-safe behavior.

Each transition should be tested for its triggering condition, expected response, and safe handling of invalid transitions.

E. Conclusion

Combining BVA and state transition testing with safety requirements, fault injection, and independent verification improves test completeness. Error Guessing remains useful as a supplementary method, but it should not replace structured testing in a safety-critical braking system.

10. Correcting a CFG with Missing Else Branches

A. Introduction

A Control Flow Graph represents the possible execution flow of a program. If the else branches of nested if statements are omitted, the CFG becomes incomplete. As a result, error-handling paths may remain untested even when the executed statements achieve 100% statement coverage.

B. Errors in the Existing CFG

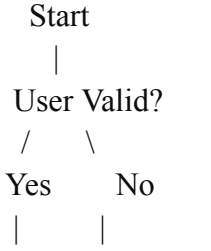
- False branches of decision nodes are missing.
- Nested decision outcomes are not represented completely.
- Error-handling statements may not have corresponding CFG nodes.
- Execution edges do not represent all possible control transfers.
- Branch coverage is incorrectly assumed from statement coverage alone.

C. Root Causes

The main causes are incomplete source-code analysis and focusing only on the successful execution path. A tester may execute every statement in the main flow but fail to exercise the false outcomes of decisions. This leaves important error paths unverified.

D. Corrected CFG and Independent Paths

Consider the following example:



```
  Password OK?  Error 1
    /    \      |
  Yes      No    |
   |       |     |
Success   Error 2 |
    \       |   /
          End
```

The independent paths are:

- Path 1: Start → User valid → Password correct → Success → End.
- Path 2: Start → User valid → Password incorrect → Error 2 → End.
- Path 3: Start → User invalid → Error 1 → End.

Test cases should include valid credentials, an invalid password for a valid user, and a nonexistent or invalid user.

E. Conclusion

A complete CFG must represent both true and false branches of every decision, including nested conditions. Testing all independent paths improves branch coverage, verifies error handling, and reduces the risk of defects remaining hidden in alternative execution routes.